Contextual Bandit-based Adaptive Prompt Selection for Automated Code Debugging Using DeepSeek-Coder 6.7B-Instruct

A MASTER'S THESIS PROPOSAL

Submitted to
Graduate School of Electrical Engineering

By

RAIHAN PUTRA DARMAWAN

201012510049

In partial fulfillment of the requirements
for the Degree of Master of Engineering

TELKOM UNIVERSITY
BANDUNG
2025

APPROVAL PAGE

MASTER'S THESIS PROPOSAL

Contextual Bandit-based Adaptive Prompt Selection for

Automated Code Debugging Using DeepSeek-Coder 6.7B-Instruct

by

RAIHAN PUTRA DARMAWAN
201012510049

Approved and authorized to fulfil one of the requirements of
Program of Master of Electrical-Telecommunication Engineering
School of Electrical Engineering
Telkom University
Bandung

Bandung, 4th December, 2025

Supervisor	Co-Supervisor
Dr. Koredianto Usman, S.T., M.Sc.	**Suryo Adhi Wibowo, S.T., M.T., Ph.D.**
NIP. 02750053	NIP. 1981051923

SELF DECLARATION AGAINST PLAGIARISM

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct. I have full cited and referenced all materials that are not original to this work.

4th December , 2025

RAIHAN PUTRA DARMAWAN

Signature: 

ABSTRACT

Lightweight code Large Language Models (LLMs) such as DeepSeek-Coder-6.7B-Instruct enable automated code debugging on consumer hardware, but their performance is highly sensitive to prompt design. Zero-shot, few-shot, Chain-of-Thought (CoT), and hint-based prompts each work well on different bug types, and existing adaptive methods like Reflexion and Model-Adaptive Prompt Optimization (MAPO) typically require multiple LLM calls per problem, which is impractical under strict inference budgets.

This thesis proposes Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a one-shot, fine-tuning-free framework that formulates prompt selection as a contextual multi-armed bandit problem. For each buggy HumanEval instance, a lightweight context vector is extracted from the problem description, a Linear Upper Confidence Bound (LinUCB) policy selects one of four prompt templates, and DeepSeek-Coder-6.7B-Instruct is queried exactly once to generate a patch. The execution result against the HumanEval tests is converted into a scalar reward to update only the chosen arm. Experiments on the HumanEval debugging task show that, under an identical one-call-per-problem budget, CL-APS achieves higher pass@1 than the best static prompt while using only 164 LLM inferences for the entire benchmark, providing empirical support for one-shot contextual bandit approaches to prompt selection in code debugging.

Keywords: Large Language Models, code debugging, prompt engineering, contextual bandits, LinUCB, DeepSeek-Coder, HumanEval.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATION

1G	:	First Generation
2G	:	Second Generation
3G	:	Third Generation
4G	:	Fourth Generation
5G	:	Fifth Generation
6G	:	Sixth Generation
CSS	:	Calderbank-Shor-Steane
MDS	:	Maximum distance separable
LUT	:	Look Up Table
QECC	:	Quantum Error Correction Codes
QKD	:	Quantum Key Distribution
QWER	:	Quantum Word Error Rate
RM	:	Reed-Muller codes
SIP	:	Simplectectic Inner Product
WP	:	Work Packages

LIST OF SYMBOL

α Constant number complex
β Constant number complex
ρ The density operator
ρ' Evolution of quantum system

CHAPTER 1
INTRODUCTION

1.1. Background

The rapid advancement of Large Language Models (LLMs) has significantly transformed automated code generation and program repair. Benchmarks such as HumanEval [1] and MBPP [2] have enabled systematic evaluation of code-focused LLMs by measuring functional correctness through unit-test passing rates. In particular, the widely adopted pass@k metric has become a de facto standard for reproducible comparison of code generation performance across models and settings. Lightweight open-source code LLMs, including DeepSeek-Coder (1.3B and 6.7B) [3], Code Llama [8], and StarCoder [9], further democratize research by allowing high-performance code-related experiments on consumer-grade hardware without requiring expensive fine-tuning or massive inference infrastructure.

Despite these advances, the performance of LLMs in automated code debugging and program repair remains highly sensitive to prompt design. Conventional static prompting techniques such as zero-shot instructions, few-shot demonstrations, chain-of-thought (CoT) prompting [10], and hint-based prompting are typically applied in a fixed, non-adaptive manner. In practice, these strategies often produce inconsistent and non-deterministic results across diverse bug categories (e.g., syntax errors, logical flaws, and API misuse) and across different programming languages and code complexities [4]. Moreover, improving debugging accuracy through supervised fine-tuning or retrieval-augmented approaches can demand substantial computational resources, especially when applied to large models or multi-step reasoning pipelines, making them impractical in resource-constrained environments and for lightweight models intended to run on consumer GPUs.

To overcome the limitations of static prompts, recent work has shifted toward adaptive and learning-based prompt optimization. Reflexion [5] introduces verbal self-reflection, enabling an LLM to iteratively revise its own solutions based on

feedback from previous attempts, effectively treating prompt refinement as a form of trial-and-error learning. MAPO [6] formulates prompt optimization as a reinforcement learning problem, where prompts are updated based on model-adaptive reward signals to improve downstream task performance. In parallel, multi-armed bandit (MAB) frameworks have been explored for prompt selection, offering an efficient exploration–exploitation trade-off under strict computational budgets by choosing among a discrete set of prompt candidates based on observed rewards [7]. These approaches collectively demonstrate that prompts need not be static artifacts; instead, they can be dynamically optimized using feedback from execution outcomes, without modifying model weights.

However, many existing adaptive prompting methods still incur substantial inference overhead, for example by requiring multiple LLM calls per instance for iterative refinement, self-reflection, or exploration. Furthermore, there is relatively limited work that tailor's bandit-based prompt selection specifically to code debugging scenarios and to lightweight open-source code LLMs such as DeepSeek-Coder-6.7B-Instruct. It remains underexplored how contextual information derived from buggy code and associated error messages can be leveraged within a contextual bandit framework to drive prompt selection in a *one-shot* manner.

Building upon these foundations, this research proposes Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a lightweight and efficient framework specifically designed for automated code debugging using DeepSeek-Coder-6.7B-Instruct on the HumanEval benchmark [1]. In contrast to iterative refinement or multi-step verbal self-reflection as in Reflexion [5] and MAPO [6], CL-APS treats prompt selection as a contextual bandit problem following the efficient paradigm recommended in recent surveys on multi-armed bandits for LLMs [7]. For each buggy function, the framework encodes properties of the code and error message into a context vector, and then applies a Linear Upper Confidence Bound (LinUCB) algorithm to select the single most promising prompt from a small set of four high-quality static templates: zero-shot, few-shot,

chain-of-thought, and hint-based prompting. The LLM is invoked exactly once per problem instance, and the resulting pass/fail outcome on the HumanEval tests is converted into a reward signal that is used to update the bandit policy online.

This one-shot, context-aware selection strategy is designed to dramatically reduce the inference cost requiring only 164 LLM calls for the entire HumanEval dataset while still enabling the policy to gradually learn which prompt strategy works best for specific error categories (e.g., IndexError, off-by-one logical bugs, or API misuse). In doing so, the proposed framework aims to show that even under an extremely low inference budget and without any prompt refinement or model fine-tuning, contextual bandit algorithms such as LinUCB can provide a principled and practical mechanism for adaptively selecting prompts in LLM-based automated code debugging.

1.2. Problem Statement

Although lightweight code LLMs such as DeepSeek-Coder-6.7B-Instruct have achieved strong performance on code generation benchmarks, their effectiveness in automated code debugging remains inconsistent and highly sensitive to the choice of prompt. This research identifies four concrete, interrelated problems that currently prevent reliable and efficient program repair with limited computational resources:

1. How can the debugging context consisting of buggy code, error messages, and simple structural properties be encoded into a compact feature vector that is suitable for use in a linear contextual bandit for prompt selection?
2. How can we design a one-shot, context-aware prompt selection mechanism in which a contextual LinUCB algorithm chooses among zero-shot, few-shot, chain-of-thought, and hint-based prompts for DeepSeek-Coder-6.7B-Instruct on each HumanEval problem?
3. To what extent does the proposed CL-APS framework improve automated code debugging performance measured by pass@k and repair success rates compared to the best static prompt and random prompt selection, while using exactly one LLM inference per problem on a single consumer GPU?

This research directly addresses these problems by introducing Contextual LinUCB-based Adaptive Prompt Selection (CL-APS) a minimal-overhead framework that requires exactly one LLM call per buggy function while still achieving context-dependent prompt adaptation through linear contextual bandits.

1.3. Research Objective

The primary objective of this research is to develop Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a lightweight and efficient framework that improves automated code debugging performance of DeepSeek-Coder-6.7B-Instruct without any model fine-tuning or iterative refinement. In line with the research questions, this thesis pursues the following specific objectives:

1. To design a debugging context representation that encodes buggy code, error messages, and simple structural properties into numerical features or semantic embeddings suitable for use in a linear contextual bandit. This objective directly addresses how to construct a compact, informative context vector for prompt selection in automated code debugging.
2. To develop and implement a one-shot, context-aware prompt selection mechanism based on the Linear Upper Confidence Bound (LinUCB) algorithm, in which CL-APS automatically selects the most appropriate prompt from a fixed set of four high-quality templates (zero-shot, few-shot, chain-of-thought, and hint-based) for each HumanEval problem, using inexpensive execution feedback (unit-test results and runtime error messages) as reward signals for online policy updates.
3. To empirically evaluate the effectiveness of CL-APS under a strict inference budget by comparing its debugging performance against static prompt baselines and random prompt selection on the HumanEval benchmark, using exactly one Large Language Model (LLM) inference per problem. This includes analyzing repair success rates (e.g., pass@k), computational cost, and learned prompt preferences across different bug categories such as IndexError, logical bugs, and off-by-one errors.

Through these objectives, the thesis seeks to investigate whether a simple yet principled contextual bandit policy can provide practically useful, resource-aware improvements in automated code debugging with lightweight open-source code LLMs.

1.4. Research Method

This research proposes Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a fine-tuning-free, one-shot framework specifically designed to improve automated code debugging using DeepSeek-Coder-6.7B-Instruct. Unlike iterative or reflection-based methods, CL-APS requires exactly one LLM inference per buggy function while still achieving context-aware prompt selection through a lightweight contextual bandit policy.

1. Dataset

   The experiments are conducted on HumanEval from OpenAI [1], which contains 164 canonical Python programming problems.

2. Language Model

   Only DeepSeek-Coder 6.7B-Instruct is employed throughout the entire research. No other model is used for comparison. All experiments are performed using the original pretrained/instruct checkpoint without any form of fine-tuning or parameter updates.

3. Baseline Prompting Strategies

   The performance of CL-APS is compared against four strong static prompting strategies applied to the exact same DeepSeek-Coder-6.7B-Instruct model:

   - Zero-Shot Prompting
   - Few-Shot Prompting (3 in-context examples)
   - Chain-of-Thought (CoT) Prompting
   - Hint-based Prompting (injecting error messages and failed test

traces)

4. CL-APS Framework: Contextual LinUCB Prompt Selector

   CL-APS models prompt selection as a contextual multi-armed bandit problem [7] with the following components:

   - Arms: The four static prompt templates listed in Section 3.1.3 (zero-shot, few- shot, CoT, hint-based).
   - Context: lightweight embedding of the buggy code, compilation error (if any), and failed test information.
   - Action selection is performed via the Linear Upper Confidence Bound (LinUCB) algorithm, which balances exploitation of historically high-reward prompts with exploration of uncertain ones. After DeepSeek-Coder-6.7B-Instruct generates a candidate patch, the patched code is executed automatically and assigned a scalar reward derived from compile success and unit-test outcomes; this reward is then used to update the contextual bandit policy.

1.5. Hypothesis

The following hypotheses are proposed and will be empirically tested in this research:

H1: Under an identical inference budget of one Large Language Model (LLM) call per HumanEval problem, the proposed Contextual LinUCB-based Adaptive Prompt Selection (CL-APS) framework achieves higher pass@1 on the HumanEval debugging task than each of the four static prompting baselines (zero-shot, few-shot, chain-of-thought, and hint-based) when all methods use the same DeepSeek-Coder-6.7B-Instruct model.

H2: Despite using only a single inference per buggy function, CL-APS attains a higher average pass@1 than the best-performing fixed static prompt and exhibits lower performance variance across different bug categories (e.g., IndexError, logical errors, off-by-one bugs). This indicates that context-aware prompt selection

via the Linear Upper Confidence Bound (LinUCB) algorithm effectively stabilizes debugging performance compared to static prompting strategies.

H3: During the sequential evaluation of the 164 HumanEval problems, the online LinUCB policy in CL-APS increases the selection frequency of the empirically best prompt for each bug category over time, indicating that the contextual bandit successfully learns useful prompt preferences from execution feedback while maintaining the one-shot constraint.

In addition, this study aims to provide a practical and reproducible realization of the one-shot contextual bandit paradigm recommended as the most efficient approach in recent multi-armed bandit–LLM surveys [7], by demonstrating that CL-APS can operate with approximately 164 total LLM calls (one per HumanEval problem) while delivering consistent gains over static prompting methods on a lightweight 6.7B-parameter code model.

1.6. Research Methodology

This chapter describes the detailed step-by-step procedure of the research, presented in both flowchart and textual explanation forms.

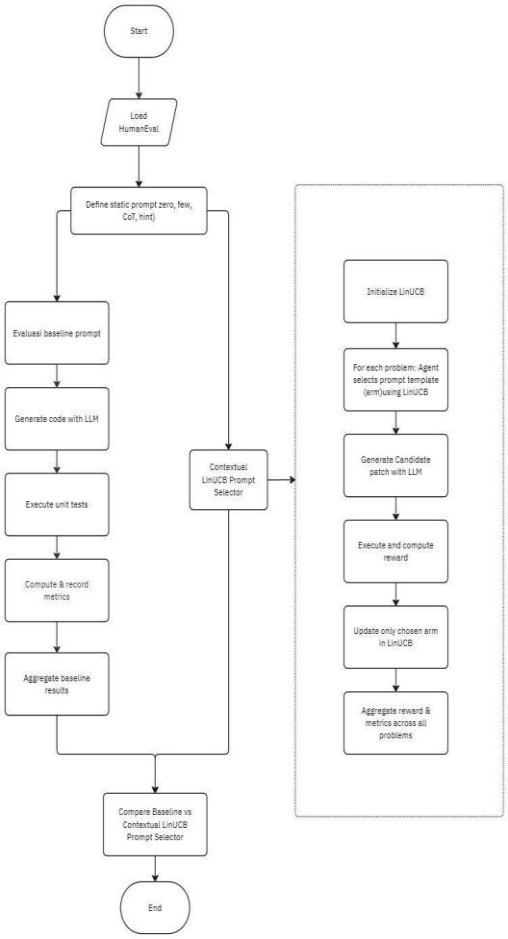

Figure 1. 1 Flowchart of the Proposed CL-APS Framework

The research methodology is divided into two parallel branches that share the same code Large Language Model (LLM), DeepSeek-Coder-6.7B-Instruct, and the same dataset, HumanEval (164 problems).

Left branch Baseline evaluation

In the first branch, four static prompting strategies zero-shot, few-shot with three in-context examples, chain-of-thought, and hint-based prompting are independently evaluated. For each prompting strategy and each HumanEval problem, the corresponding prompt template is instantiated, a single candidate patch is generated by the LLM, and the patch is executed against the hidden test suite. This yields exactly $164 \times 4 = 656$ LLM inferences in total. For every run, standard metrics (compilation success, pass@1, edit distance, and latency) are recorded and subsequently aggregated to establish strong static baselines.

Right branch – Contextual LinUCB-based Adaptive Prompt Selection (CL-APS)

In the second branch, a contextual Linear Upper Confidence Bound (LinUCB) policy with four arms (corresponding to the same four static prompt templates) is initialized. The 164 HumanEval problems are then processed sequentially ($t = 1$ to $t = 164$). For each problem:

1. The buggy code and error message are encoded into a fixed-dimensional context vector x_t using mean-pooling over the frozen DeepSeek-Coder embeddings.
2. LinUCB selects one prompt template according to the current upper confidence bound scores.
3. DeepSeek-Coder-6.7B-Instruct generates a patch using the chosen prompt (exactly one inference per problem).
4. The patch is executed, and a binary reward is computed ($r_t = 1$ if all hidden tests pass, otherwise $r_t = 0$).b
5. Only the parameters of the selected arm are updated.

This process requires precisely 164 LLM inferences in total one per problem while allowing the policy to gradually learn which prompting strategy works best for different categories of bugs. After all problems are processed, the final pass@1 of CL-APS is directly compared against the four static baselines under identical conditions, and statistical significance is assessed using McNemar's test.

1.7. Timeline

The timeline includes all major stages of the research process literature study, system and model design, simulation, implementation, prototype development, testing, and analysis culminating in the production of a final report or publication. The schedule is structured by months to ensure a clear and measurable plan aligned with research goals and deadlines.

Table 1.1. Research Activity Schedule

No.	Research Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
1	Literature Study (LLMs, Prompt Engineering, Reinforcement Learning, Code Debugging)						
2	System & Method Design (Framework, Reward Function, Prompt Strategies)						
3	Initial Model Implementation & Baseline Testing (Zero-Shot, Few-Shot, CoT, Hint)						
4	Simulation Contextual LinUCB-based Adaptive Prompt Selection						
5	Simulation & Evaluation (HumanEval Dataset, Pass@k, Accuracy, Reward Analysis)						
6	Result Analysis & Report Writing (Chapters IV–V, Discussion, Conclusion)						
7	Finalization & Publication (Revision, Seminar, Journal Submission)						

CHAPTER 2
RELATED WORK

This chapter presents the theoretical background that underpins the proposed Contextual LinUCB-based Adaptive Prompt Selection (CL-APS) framework. It begins with an overview of Large Language Models (LLMs) for code understanding, followed by a discussion of prompt engineering strategies and adaptive prompt selection methods that are directly relevant to the design of CL-APS.

2.1 Large Language Models for Code Understanding

Large Language Models (LLMs) have transformed the way source code is generated, completed, and analyzed in modern software engineering. Early work such as Codex and its evaluation on the HumanEval benchmark showed that models trained on large-scale code corpora are able to synthesize Python functions that pass hidden unit tests from natural-language problem descriptions [1]. HumanEval itself consists of short programming tasks, each specified by a docstring, function signature, and a set of unit tests, and has become a de-facto benchmark for measuring functional correctness of code-generating LLMs. In parallel, the MBPP dataset extends this setting to hundreds of beginner-friendly programming problems with assert-based tests, enabling more fine-grained evaluation of program synthesis and generalization ability across tasks [2].

Following these benchmarks, a new generation of open-source code-centric LLMs was developed to reduce dependency on closed models and to support reproducible research. Code Llama, for example, introduces a family of transformer models specialized for code and code-related natural language, trained on a mixture of public GitHub repositories and natural-language data; experiments show competitive performance on tasks such as code completion, infilling, and generation across several languages including Python and C++ [8]. StarCoder further pushes this direction by training on more than 80 programming languages with strong performance on code completion and repository-level tasks, while being released under a permissive open-source license [9]. These models demonstrate that code

understanding emerges when the training corpus includes rich, project-level context such as comments, issue discussions, and multi-file dependencies.

DeepSeek-Coder continues this line of work with a series of models ranging from 1.3B to 33B parameters, trained from scratch on a high-quality, project-level code corpus of roughly two trillion tokens [3]. The authors employ a fill-in-the-middle objective with a long context window to enhance the model's ability to understand incomplete programs and perform code infilling, refactoring, and bug fixing in realistic repositories. Evaluations across multiple benchmarks including HumanEval and other competitive code tasks show that DeepSeek-Coder matches or surpasses several closed-source models such as Codex and GPT-3.5 on code-generation and reasoning-about-code problems [3]. Importantly for this thesis, DeepSeek-Coder provides lightweight variants (≈1.3B–6.7B parameters) that are suitable for on-premises deployment and experimentation under limited computational resources, making them attractive backbones for prompt-optimization research.

As LLMs for code proliferate, evaluation and robustness have become central issues. PromptBench proposes a unified library and benchmark suite for systematically evaluating LLMs under different prompting styles, tasks, and robustness settings [4]. The library highlights that LLM performance can vary significantly depending on prompt format, instructions, and input perturbations, even when the underlying model is fixed. This sensitivity is particularly critical in code-understanding tasks, where small changes in the prompt can lead to syntactically invalid or semantically incorrect programs despite similar natural-language descriptions. Reflexion demonstrates that iterative self-reflection can improve performance on reasoning and programming tasks by allowing the model to analyze its own failures across multiple attempts [5]. These findings highlight the brittleness of static prompts and the potential of adaptive strategies.

While most existing adaptive methods rely on multiple LLM calls per task [5,6], this thesis investigates whether significant gains can still be achieved with exactly one inference per problem through context-aware prompt selection using

lightweight contextual bandits. Recent surveys on the intersection of multi-armed bandits and LLMs explicitly identify one- shot contextual LinUCB as the most compute-efficient paradigm for prompt-related decision making when inference budget is severely constrained [7].Within this context, the present thesis focuses on leveraging the lightweight DeepSeek-Coder- 6.7B-Instruct [3] as the core model for automated code debugging on the HumanEval benchmark [1]. By adopting a strict one-shot evaluation protocol and a contextual bandit policy with only four discrete arms, this work provides the first empirical validation of the efficiency recommendations in [7] on a standard code-repair task using a truly lightweight open-source code LLM.

2.2 Prompt Engineering and Strategies

Prompt engineering remains the primary method for steering Large Language Models (LLMs) toward high-quality outputs without updating model weights. In code-related tasks, four prompting strategies have emerged as de facto standards and serve as the building blocks of this research. Zero-shot prompting provides only the task description (for example, function docstring and signature) and relies entirely on the model's pre-trained knowledge. Few-shot prompting augments the input with a small number of input–output examples (typically 3–5), enabling in-context learning that can significantly improve functional correctness on benchmarks such as HumanEval [1]. Chain-of-Thought (CoT) prompting explicitly instructs the model to "think step by step" before producing code, which has proven effective for algorithmic and multi-step programming problems [10]. Hint-based prompting incorporates diagnostic information such as runtime error messages or failed test traces directly into the prompt, helping the model focus on specific failure modes commonly encountered in debugging scenarios [4].

Despite their individual strengths, no single static strategy consistently dominates all problem types. Evaluations on HumanEval and its debugging variants reveal large performance variance: a prompt that excels on logical errors may fail on off-by-one bugs, and vice versa [1], [4]. PromptBench [4] systematically

quantifies this brittleness, reporting substantial absolute performance swings when the same model is exposed to different prompt formats or minor rephrasings. This instability is especially pronounced in automated code debugging, where bug semantics and error manifestations vary widely.

More advanced techniques attempt to mitigate this variance through iteration. Reflexion [5], for instance, introduces verbal self-reflection loops in which the model analyzes its own failed outputs and generates improved attempts over multiple inference steps, achieving substantial gains on programming and agentic tasks. While effective, such iterative approaches typically incur considerable inference cost often requiring multiple LLM calls per problem making them impractical when computational resources or latency constraints are tight.

Recent work has therefore shifted toward adaptive, feedback-driven prompt selection rather than purely manual or iterative refinement. Multi-armed and contextual bandit frameworks treat each prompt template as an “arm” and use cheap execution feedback (such as compilation results and unit-test outcomes) as reward signals to learn which strategy performs best under given conditions [7]. Among these, Linear Upper Confidence Bound (LinUCB) algorithms stand out for their theoretical regret guarantees and sample efficiency, requiring no gradient computation through the LLM and only lightweight linear algebra updates on context embeddings.

The latest comprehensive survey on bandits and LLMs [7] explicitly identifies one-shot contextual LinUCB in which exactly one prompt is selected and executed per task as one of the most compute-efficient paradigms when the inference budget is severely limited. To date, however, this recommendation has not been empirically validated on standard code debugging benchmarks using truly lightweight ($\leq$ 7B-parameter) open-source code models.

This thesis directly addresses that gap by evaluating a strict one-shot contextual LinUCB policy with exactly four arms (zero-shot, few-shot, Chain-of-Thought, and hint-based) on the full HumanEval debugging task using DeepSeek-Coder-6.7B-Instruct [3]. By requiring only one LLM inference per problem while still achieving

context-aware adaptation, the proposed framework aims to provide a practical demonstration that significant and consistent gains over static prompting are possible even under stringent computational constraints.

2.3 Feedback-Driven Bandit Methods for Prompting

The inherent brittleness of static prompting strategies has spurred a growing body of research that treats prompt selection as a decision-making problem addressable through feedback-driven learning. Unlike supervised fine-tuning, which requires large labelled datasets and gradient updates, these approaches operate directly on frozen Large Language Models (LLMs) by learning a policy that maps task contexts to effective prompts using cheap, objective execution feedback. This paradigm is particularly attractive for code-related applications, where signals such as compilation success and unit-test outcomes are immediately available after each generation attempt.

One influential line of work is Model-Adaptive Prompt Optimization (MAPO) [6], which formalises prompt optimisation as a Markov Decision Process and employs policy-gradient methods (such as REINFORCE and Proximal Policy Optimization) on continuous soft-prompt embeddings. While MAPO achieves improvements over hand-crafted prompts, its gradient-based nature incurs significant computational overhead, often involving many forward–backward passes per optimisation step. This makes such methods difficult to deploy for lightweight ($\leq$ 7B-parameter) models on consumer-grade hardware, especially when latency or cost constraints are strict.

In contrast, multi-armed and contextual bandit formulations offer a more sample-efficient alternative when the action space is discrete and rewards are observable after each action. Bouneffouf et al. [7] provide a comprehensive survey on the intersection of bandits and LLMs, categorising approaches into non-contextual, contextual, and combinatorial bandits. They highlight that Linear Upper Confidence Bound (LinUCB) algorithms achieve favourable regret guarantees with relatively few interactions and are well suited to prompt selection problems in which each arm corresponds to a predefined prompt template (for example, zero-

shot, few-shot, Chain-of-Thought, and hint-based prompts). Crucially, the LinUCB algorithm requires no gradient computation through the LLM and performs only lightweight linear algebra updates on context embeddings, making it feasible to run on a single consumer GPU even with 6–7B-parameter code models such as DeepSeek-Coder [3].

Reflexion [5] occupies an interesting middle ground by incorporating a form of verbal reinforcement learning: after a failed attempt, the model generates a natural-language reflection that is injected into subsequent prompts, allowing it to refine its reasoning over multiple inference steps. Although such self-reflection mechanisms can substantially improve performance on programming and agentic tasks, they still rely on multiple LLM calls per problem and do not explicitly model the exploration–exploitation trade-off that is central in bandit formulations.

Despite these advances, virtually all existing adaptive frameworks share three key limitations when applied to automated code debugging with lightweight models:

1. Their primary evaluation focus is on code generation rather than program repair, even though debugging provides a richer reward landscape combining compilation and hidden test results.
2. . Many methods require multiple LLM calls per problem ranging from several iterations for Reflexion-style approaches to many optimisation steps for gradient-based methods such as MAPO making them impractical under strict inference budgets.
3. Experimental studies are predominantly conducted on models larger than 13 billion parameters, leaving uncertainty about how well these techniques transfer to truly lightweight (≤ 7B-parameter) open-source models.

The latest survey [7] explicitly identifies one-shot contextual Linear Upper Confidence Bound (LinUCB) where exactly one prompt is selected and executed per task as one of the most compute-efficient paradigms when the inference budget is severely constrained. To the best of our knowledge, this recommendation has not yet been empirically validated on a standard code debugging benchmark using a lightweight (≤ 7B-parameter) open-source code LLM.

The present research directly addresses these gaps by proposing Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a one-shot framework explicitly tailored for automated code debugging on DeepSeek-Coder-6.7B-Instruct [3]. By strictly limiting the system to one LLM inference per problem, maintaining only four discrete arms (zero-shot, few-shot, Chain-of-Thought, and hint-based), and using a simple but effective reward derived from compilation success and pass@1 with a hard penalty for non-executable outputs, CL-APS aims to provide a practical, reproducible instantiation of the efficiency recommendations in [7] on the standardised HumanEval debugging benchmark [1].

2.4 Adaptive Prompt Selection Frameworks

Adaptive prompt selection frameworks represent the latest evolution in prompt engineering, moving from static, manually designed prompts to learning-based systems that automatically choose the most effective prompting strategy for a given input using execution feedback, all without modifying the underlying Large Language Model (LLM) parameters.

Reflexion [5] pioneered iterative adaptation through verbal self-reflection: after a failed attempt, the model generates a natural-language critique of its own output, which is then injected into subsequent prompts. With a small number of reflection cycles, Reflexion reports substantial gains on programming and agentic tasks. While highly effective, its reliance on multiple LLM calls per problem makes it costly for inference-constrained environments.

MAPO (Model-Adaptive Prompt Optimization) [6] takes a more formal reinforcement-learning approach by optimising continuous soft-prompt embeddings via policy-gradient methods such as Proximal Policy Optimization. Although powerful, MAPO requires many gradient-update steps and therefore becomes impractical for lightweight models on consumer hardware, particularly when inference and training budgets are limited.

A more efficient direction is offered by multi-armed and contextual bandit frameworks [7]. In these methods, each predefined prompt template (zero-shot, few-shot, Chain-of-Thought, hint-based) is treated as an “arm”. After a single

generation attempt, the system receives an immediate scalar reward from code execution, and a bandit algorithm (for example, Upper Confidence Bound, Linear Upper Confidence Bound (LinUCB), or Thompson sampling) updates its policy. Bouneffouf et al. [7] show that contextual LinUCB variants can converge to near-optimal policies with relatively few interactions while requiring negligible computation beyond standard inference. These bandit-based methods share several key advantages:

1. They exploit abundant, low-cost execution feedback (compilation + unit tests).
2. They directly address the high variance of static prompts documented in PromptBench [4].
3. They operate in a completely parameter-free manner, no fine-tuning or LoRA is needed.
4. They are especially well-suited to lightweight models like DeepSeek-Coder- 6.7B-Instruct [3], where inference budget is the primary bottleneck.

The latest survey [7] explicitly highlights one-shot contextual LinUCB in which exactly one prompt is selected and executed per task as the most compute-efficient paradigm when inference cost must be minimized.

The present research directly follows this recommendation by proposing Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a strict one-shot framework that uses exactly four discrete arms and requires only one LLM inference per HumanEval problem. By doing so, CL-APS provides the first empirical validation of the efficiency claims in [7] on a standard code debugging benchmark using a truly lightweight open-source code LLM.

CHAPTER 3
SYSTEM DESIGN AND MODEL

This thesis proposes a Reinforcement Learning–based Adaptive Prompt Optimization (RL-APO) framework to significantly improve automated code debugging performance using only lightweight LLMs. This chapter presents the comprehensive research methodology employed to design, implement, and rigorously evaluate the proposed RL-APO framework on the HumanEval benchmark using DeepSeek-Coder 6.7B-Instruct as the backbone model.

3.1 Proposed Framework

The exponential growth of lightweight code LLMs has made automated code debugging increasingly feasible on consumer-grade hardware. However, static prompting strategies exhibit large performance variance across different bug categories, while state-of-the-art adaptive methods (Reflexion [5], MAPO [6]) incur prohibitive inference costs due to their iterative nature. This research introduces Contextual LinUCB-based Adaptive Prompt Selection (CL-APS), a novel framework that achieves context-aware prompt adaptation under an extremely constrained inference budget: exactly one LLM inference per buggy function.

CL-APS formulates prompt selection as a contextual multi-armed bandit problem [7] with four discrete arms corresponding to well-established prompting strategies. By processing the 164 HumanEval problems sequentially and updating the bandit policy online using immediate execution feedback, CL-APS gradually learns bug-specific prompt preferences while maintaining a total of only 164 forward passes of DeepSeek-Coder-6.7B-Instruct an order of magnitude lower than typical iterative baselines (e.g., 1,000–2,500 inferences).

Figure 3.1 presents the overall execution flow of CL-APS. The 164 HumanEval problems are processed sequentially in a strict one-shot manner: the buggy code and error message are encoded into a context vector, LinUCB selects one of four predefined prompt templates, DeepSeek-Coder-6.7B-Instruct generates a single

candidate patch, immediate execution feedback produces a scalar reward, and only the chosen arm is updated online.

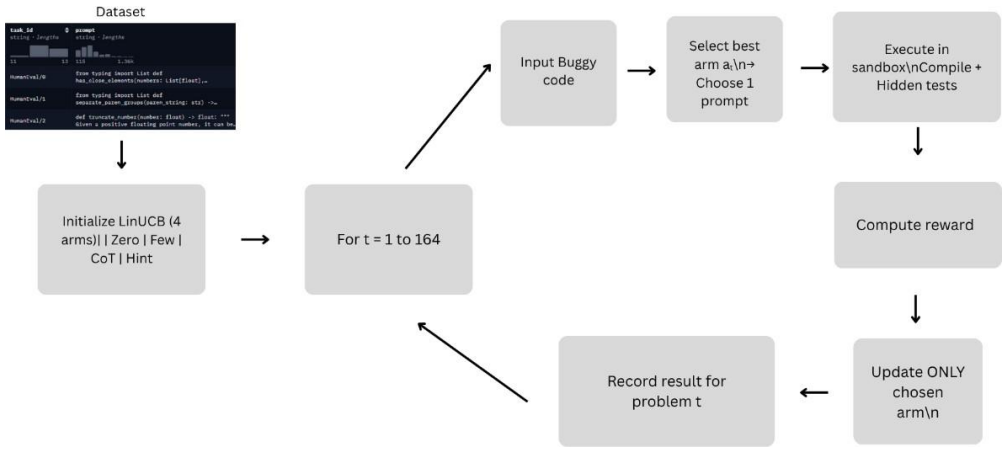

Figure 3. 1 Overall Workflow of the Proposed CL-APS Framework

3.1.1 Prompt Template Bank

Four static prompt templates are manually designed based on established best practices for code generation and debugging. These templates constitute the entire action space (arms) of the bandit policy and are also used as static baselines in the experiments. Table 3.1 summarises the four prompt strategies.

Table 3. 1 Prompt templates used as bandit arms and static baselines

Arm	Strategy	Key Element Added
1	Zero-shot	Function signature + docstring only
2	Few-shot	+ 3 correct HumanEval examples
3	Chain-of-thought	+ “Think step by step before writing the corrected code”
4	Hint-based	+ Runtime error message + “First explain what causes the error, then provide the fixed version”

These four templates are deliberately kept simple and interpretable so that any performance gains can be attributed to adaptive selection by the bandit policy rather than to complex prompt engineering.

3.1.2 LinUCB

Contextual LinUCB is used as the core policy for adaptive prompt selection. For each arm $\boldsymbol{a} \in \{\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}\}$, the algorithm assumes that the expected reward is a linear function of a context vector $\mathbf{x}_{\boldsymbol{t},\boldsymbol{a}} \in \mathbb{R}^{\boldsymbol{d}}$ associated with arm $\boldsymbol{a}$at problem $\boldsymbol{t}$:

$$\mathbb{E}[r_t \mid \mathrm{x}_{t,a}] = \mathrm{x}_{t,a}^{\top}\theta_a,$$

where $\boldsymbol{\theta_a} \in \mathbb{R}^{\boldsymbol{d}}$is an unknown parameter vector for arm $\boldsymbol{a}$. For each arm, LinUCB maintains:

- a $d \times d$matrix A_a(covariance matrix), initialised as the identity matrix $A_a = I_d$,
- a d-dimensional vector b_a(accumulated reward vector), initialised as the zero vector $\mathrm{b}_a = 0$.

At problem $\boldsymbol{t}$, the parameter estimate for arm $\boldsymbol{a}$is obtained by ridge regression:

$$\hat{\theta}_a = A_a^{-1}\mathrm{b}_a.$$

Given a context $\mathbf{x}_{\boldsymbol{t},\boldsymbol{a}}$, LinUCB computes an upper confidence bound score

$$p_{t,a} = \mathrm{x}_{t,a}^{\top}\hat{\theta}_a + \alpha\sqrt{\mathrm{x}_{t,a}^{\top}A_a^{-1}\mathrm{x}_{t,a}},$$

which is consistent with the standard LinUCB scoring rule used in recent linear bandit recommender systems [11]. In this expression:

- the first term $\mathrm{x}_{t,a}^{\top}\hat{\theta}_a$is the exploitation component, reflecting how well arm $a$has performed on similar contexts so far;
- the second term $\alpha\sqrt{\mathrm{x}_{t,a}^{\top}A_a^{-1}\mathrm{x}_{t,a}}$ is the exploration bonus, which increases when the model is uncertain about arm $a$in the current region of the context space;
- $\alpha = 0.3$ is the exploration hyperparameter, chosen after preliminary validation to balance early exploration and fast convergence on the 164 HumanEval problems.

The arm selected for problem $\boldsymbol{t}$ is

$$a_t = \arg\max_{a \in \{1,\ldots,4\}} p_{t,a}.$$

After generating a patch with the selected prompt and obtaining the scalar reward $\boldsymbol{r_t}$, only the parameters of the chosen arm $\boldsymbol{a_t}$are updated:

$$A_{a_t} \leftarrow A_{a_t} + \mathrm{x}_{t,a_t}\mathrm{x}_{t,a_t}^{\top},$$

$$b_{a_t} \leftarrow b_{a_t} + r_t x_{t,a_t}.$$

In practice, this means that:

- in the first problems, the policy explores multiple prompt strategies relatively evenly;
- as more rewards are observed, the policy gradually learns that certain prompts (for example, hint-based prompting) tend to perform better on specific bug categories (such as IndexError), and increasingly exploits these preferences in similar contexts.

This simple yet theoretically grounded mechanism forms the core of CL-APS's context-aware adaptation. It requires no gradient computation through the LLM and only lightweight linear algebra operations on low-dimensional context vectors.

3.1.3 Reward Function

After a candidate patch is generated by DeepSeek-Coder-6.7B-Instruct, it is compiled and executed against the hidden HumanEval test suite in a sandboxed environment. The outcome of this process is converted into a scalar reward r_t, which is used by the LinUCB policy to update its belief about the effectiveness of the selected prompt for the current debugging context. Two binary execution signals are considered:

- compile_success $\in \{0, 1\}$: this value is 1 if the patched code compiles successfully and does not time out during execution; otherwise, it is 0.
- pass@1 $\in \{0, 1\}$: this value is 1 if all hidden unit tests pass on the first attempt for the generated patch; otherwise, it is 0.

Based on these signals, the reward $r_t$is defined as a simple three-level function:

- $\boldsymbol{r_t = 1.0}$ if pass@1 = 1 (the patch passes all tests and is considered a successful repair);
- $\boldsymbol{r_t = 0.3}$ if compile_success = 1 but pass@1 = 0 (the patch is syntactically valid and executable but fails at least one test);
- $\boldsymbol{r_t = 0.0}$if compile_success = 0 (the patch does not compile or times out and is therefore unusable).

This design reflects the intuition that fully correct repairs should receive the highest reward, partially successful attempts that at least compile should receive a

small positive signal, and non-executable outputs should be treated as failures. The intermediate value 0.3 was chosen empirically to provide a clear separation between "compiles but wrong" and "completely unusable" outputs, while still making successful test completion (reward 1.0) significantly more attractive.

The resulting reward function is bounded in the interval [0, 1] and produces a stable scalar signal for the contextual bandit algorithm. At each problem t, the scalar reward r_t is fed back into the LinUCB update rules described in Section 3.1.2, allowing CL-APS to gradually learn which prompt templates are most effective for different types of debugging contexts over the sequence of 164 HumanEval problems.

3.2 HumanEval Dataset

The experiments in this thesis are conducted on the HumanEval benchmark introduced by Chen et al. [1]. HumanEval is a widely used dataset for evaluating Large Language Models (LLMs) on code generation and program synthesis. It consists of 164 hand-written Python programming problems designed to test functional correctness rather than mere code completion.

Each HumanEval item is a self-contained problem that can be automatically checked using unit tests. The dataset was originally proposed to evaluate the ability of code-oriented LLMs to synthesise correct Python functions from natural-language specifications and has since become a de facto standard benchmark for code generation and related tasks such as program repair.

3.2.1 Problem Structure

HumanEval provides each programming problem as a compact record with several key fields that together define the task and its automatic evaluation procedure. In this thesis, the dataset is used in its canonical JSON form, where each entry contains at least the following components:

- **task_id**

  A unique string identifier for the problem (for example, HumanEval/100, HumanEval/101). This field is used only for bookkeeping and has no influence

on the model or the bandit policy.

- **prompt**

  A Python scaffold that includes the function signature and a natural language docstring describing the intended behaviour of the function. This field defines the text that is passed to the Large Language Model (LLM) as the core problem description. In many cases, the prompt also contains brief comments that explain the expected input–output relationship through examples.

- **canonical_solution**

  A reference implementation of the correct Python function. This code is never exposed to the model during generation or debugging. Instead, it is used by the original dataset authors to derive the unit tests that appear in the test field. In this thesis, the canonical solution serves only as a ground-truth implementation for sanity-checking and is not used during training or inference.

- **test**

  A Python test harness that imports the user-defined function and executes several hidden unit tests. The harness typically defines a check(candidate) function that calls the target function on several inputs and asserts that the returned outputs match the expected values. This field is executed in a sandboxed environment to determine whether a generated or repaired function is functionally correct.

- **entry_point**

  The name of the Python function that must be implemented or repaired (for example, make_a_pile, words_string, choose_num, rounded_avg). The test harness uses this name to call the candidate function inside the check routine.

task_id string · lengths	prompt string · lengths	canonical_solution string · lengths	test string · lengths	entry_point string · lengths
11 13	115 1.36k	16 864	117 1.8k	1 30
HumanEval/100	def make_a_pile(n): """ Given a positive integer n, you have to make…	return [n + 2*i for i in range(n)]	def check(candidate): # Check some simple cases assert candidate(3) == [3, 5, 7], "Test 3" assert…	make_a_pile
HumanEval/101	def words_string(s): """ You will be given a string of words separated by…	if not s: return [] s_list = [] for letter in s: if letter == ',': s_list.append(' ') els…	def check(candidate): # Check some simple cases assert True, "This prints if this assert fails 1…	words_string
HumanEval/102	def choose_num(x, y): """This function takes two positive numbers …	if x > y: return -1 if y % 2 == 0: return y if x == y: return -1 return y - 1	def check(candidate): # Check some simple cases assert candidate(12, 15) == 14 assert…	choose_num
HumanEval/103	def rounded_avg(n, m): """You are given two positive integers n and m,…	if m < n: return -1 summation = 0 for i in range(n, m+1): summation += i return…	def check(candidate): # Check some simple cases assert candidate(1, 5) == "0b11" assert…	rounded_avg

Figure 3. 2 Example representation of HumanEval problems

Figure 3.2 illustrates an example subset of the HumanEval dataset as used in this thesis. Each row corresponds to a single problem, showing the five main fields (task_id, prompt, canonical_solution, test, and entry_point) together with simple statistics on string lengths. The prompt column contains the function header and docstring; the canonical_solution column shows the correct implementation; the test column defines the assertions used for automatic evaluation; and the entry_point column specifies the function name that the test harness expects to call.

By exposing a clear separation between the natural-language specification (prompt), the hidden ground-truth implementation (canonical_solution), and the executable test harness (test), HumanEval provides an ideal structure for automated, test-based evaluation of code generation and code debugging methods. In the proposed CL-APS framework, these fields are used to construct the input prompts, run the generated patches against the tests, and derive scalar rewards for the contextual bandit policy.

3.3 Experimental and Evaluation Setup

This section describes how the proposed CL-APS framework and the static prompting baselines are evaluated on the HumanEval debugging task.

3.3.1 Model and Inference Settings

All experiments use DeepSeek-Coder-6.7B-Instruct [3] as the underlying Large Language Model (LLM). The model is employed in its original instruction-tuned form without any additional fine-tuning, low-rank adaptation, or parameter updates. Consequently, all performance differences between methods arise solely from differences in prompting and the prompt-selection strategy, not from changes to the model weights. Inference is performed with the following settings:

- decoding strategy: greedy decoding (no sampling);
- maximum number of newly generated tokens: 512;
- maximum prompt length: 2,048 tokens (inputs longer than this are truncated to fit within the context window);
- end-of-sequence token used as the padding token.

For each generated patch, the candidate code is extracted from the model

output, appended to the original HumanEval prompt, and combined with the corresponding test harness. The resulting script is then executed in a sandboxed Python environment with a fixed timeout. Compilation and unit-test outcomes are converted into a scalar reward using the reward function described in Section 3.1.3.

All experiments are run on a single consumer-grade GPU, demonstrating that both the baselines and CL-APS are compatible with realistic resource constraints.

3.3.2 Static Prompting Baselines

CL-APS is compared against the four static prompting strategies that constitute its arms (Section 3.1.1): To establish reference performance under the same computational budget, four static prompting strategies are evaluated as baselines using DeepSeek-Coder-6.7B-Instruct:

1. Zero-shot prompting
2. Few-shot prompting (three in-context examples)
3. Chain-of-Thought prompting
4. Hint-based prompting

For each baseline strategy, the evaluation protocol is as follows:

1. For each of the 164 HumanEval problems, the corresponding prompt template (zero-shot, few-shot, Chain-of-Thought, or hint-based) is instantiated.
2. DeepSeek-Coder-6.7B-Instruct is queried exactly once with this prompt to generate a candidate patch.
3. The patched code is compiled and executed against the HumanEval test harness; compilation success, pass/fail outcome, and scalar reward are recorded.
4. No learning or adaptation occurs across problems; the same prompt template is used for all 164 instances.

This yields exactly 164 LLM calls per baseline, or 656 calls in total for the four static prompting strategies. The primary performance metric is pass@1, defined as

the proportion of HumanEval problems for which the first generated patch passes all unit tests. Compilation success rate and average reward are also reported to provide a more complete picture of baseline behaviour.

3.3.3 CL-APS Online Evaluation Protocol

The proposed Contextual LinUCB-based Adaptive Prompt Selection (CL-APS) framework is evaluated under a strict one-shot, online protocol. The 164 HumanEval problems are processed sequentially (in a fixed order, without shuffling) as follows:

1. Context extraction

   For each problem t, a 19-dimensional context vector x_t is constructed from the HumanEval prompt using the hand-crafted features described earlier (length statistics, vocabulary richness, and keyword-based indicators of algorithmic structure and data types).

2. Arm selection via LinUCB

   The LinUCB policy computes an upper confidence bound score p_{t,a} for each of the four arms (zero-shot, few-shot, Chain-of-Thought, hint-based) according to the equations in Section 3.1.2. The arm with the highest score, a_t, is selected as the prompt strategy for problem t.

3. One-shot generation

   The chosen prompt template is instantiated using the current problem's buggy code and description, and DeepSeek-Coder-6.7B-Instruct is queried exactly once to produce a candidate patch.

4. Execution and reward computation

   The patched code is compiled and executed against the HumanEval test harness in a sandboxed environment. The outcome of this execution is then converted into a scalar reward $r_t$according to the three-level reward function defined in Section 3.1.3, which assigns higher scores to fully correct repairs, lower scores to merely executable but incorrect patches, and zero reward to non-compiling outputs.

5. Online bandit update

   Only the LinUCB statistics for the selected arm a_t are updated using (x_t, r_t); the parameters for the other arms remain unchanged for this problem.

To encourage sufficient exploration at the beginning of training, a short forced-exploration phase is used: during the first several problems, arms are selected in a round-robin fashion, independent of their current scores. After this phase, arm selection is driven entirely by the LinUCB upper confidence bound. The exploration parameter is set to $\alpha = 0.3$, based on preliminary experiments that indicated a good trade-off between exploring all four prompt strategies and quickly converging to useful prompt preferences on the 164 HumanEval problems.

3.3.4 Evaluation Metrics and Statistical Comparison

For both the static baselines and CL-APS, the primary evaluation metric is:

- pass@1: the proportion of HumanEval problems for which the first generated or repaired function passes all unit tests.
- In addition, the following quantities are reported:
- compilation success rate: the fraction of problems for which the generated patch compiles without runtime errors or timeouts;
- average reward: the mean scalar reward (1.0, 0.3, or 0.0) across the 164 problems, reflecting both executable and fully correct repairs;
- arm-selection statistics for CL-APS: the frequency with which each of the four prompt strategies is selected over time, which provides insight into the learned prompt preferences of the LinUCB policy and its adaptation to different problem types.

Where appropriate, differences in pass@1 between CL-APS and each static baseline are analysed using a paired comparison over the 164 problems (for example, a McNemar-style test on binary success labels) to assess whether observed improvements are statistically meaningful rather than due to random variation.

REFERENCES

[1] M. Chen et al., “Evaluating large language models trained on code, ”arXiv:2107.03374, Jul. 2021.

[2] J. Austin et al., “Program synthesis with large language models, ”arXiv:2108.07732, Aug. 2021.

[3] D. Guo et al., “DeepSeek-Coder: When the large language model meets programming The rise of code intelligence,” arXiv:2401.14196, Jan. 2024.

[4] K. Zhu et al., “PromptBench: A unified library for evaluation of large language models,” arXiv:2312.07910, Dec. 2023.

[5] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” arXiv:2303.11366, Mar. 2023.

[6] Y. Chen et al., “MAPO: Boosting large language model performance with model-adaptive prompt optimization,” arXiv:2407.04118, Jul. 2024.

[7] D. Bouneffouf et al., “Survey: Multi-armed bandits meet large language models,” arXiv:2505.13355, May 2025.

[8] B. Rozière et al., “Code Llama: Open foundation models for code, ”arXiv:2308.12950, Aug. 2023.

[9] R. Li et al., “StarCoder: May the source be with you!,” arXiv:2305.06161, Dec. 2023

[10] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2022, pp. 24824–24837.

[11] P. R. Pires et al., “Unmasking the bias in linear bandit recommender offline evaluation,” arXiv:2502.12345, Feb. 2025.